



Materiały dydaktyczne

Przedmiot: Programowanie obiektowe/Pracownia programowania obiektowego

Opracował: Mirosław Miciak

Ćwiczenie: C10

Tematy: Podział programu na funkcje, Biblioteki języka programowania, Proste i złożone typy danych, Zmienne, Operacje na zmiennych: wejścia, wyjścia, arytmetyczne logiczne, Operacje na typach złożonych, Operatory arytmetyczne, przypisania, porównania, Operatory logiczne i bitowe, Operatory do obsługi łańcuchów, Instrukcja warunkowa i instrukcja wyboru, Instrukcje pętli, Wykorzystanie wybranej biblioteki matematycznej, Wykorzystanie bibliotek algorytmów, Funkcje, Klasa, obiekt, metoda, pole, Projektowanie prostych klas, Zmienne obiektowe, Zakres widoczności pól klasy, Metody klasy, Konstruktor i destruktor, Konstruktor kopiujący, Metody klasy - zakres widoczności, Prosta aplikacja obiektowa: konstruktor, destruktor, Składniki statyczne klasy, Klasy dziedziczone, Klasy zaprzyjaźnione, Dziedziczenie, Hermetyzacja, Polimorfizm, Funkcje zaprzyjaźnione z klasą, Metody do obsługi elementów statycznych klasy, Klasy bazowe i pochodne, Metody wirtualne, Klasy abstrakcyjne, Szablony klas dla prostych typów liczbowych, Obsługa wyjątków, Instrukcja throw, Obsługa błędów wykonania aplikacji.

Klasy bazowe i pochodne

Programowanie obiektowe (OOP) to paradygmat programowania, który opiera się na koncepcji obiektów. Obiekty mogą zawierać dane w postaci pól (znanych również jako atrybuty lub właściwości) oraz kod w postaci metod (funkcji). Klasy bazowe i pochodne, metody wirtualne oraz klasy abstrakcyjne są kluczowymi elementami OOP, które umożliwiają tworzenie elastycznych i rozszerzalnych aplikacji.

Klasa bazowa (superklasa) to klasa, która może być dziedziczona przez inne klasy. Klasa pochodna (subklasa) dziedziczy właściwości i metody klasy bazowej, a także może dodawać własne właściwości i metody lub nadpisywać te odziedziczone.

Przykład 1. Klasy bazowe i pochodne

```
// Klasa bazowa
public class Osoba
{
    public string Imie { get; set; }
    public string Nazwisko { get; set; }

    // Metoda klasy bazowej
    public void PrzedstawSie()
    {
        Console.WriteLine($"Nazywam się {Imie} {Nazwisko}");
    }
}

// Klasa pochodna
public class Student : Osoba
{
    public string NumerIndeksu { get; set; }

    // Metoda klasy pochodnej
    public void PrzedstawSieJakoStudent()
    {

```

```

        Console.WriteLine($"Nazywam się {Imie} {Nazwisko} i jestem studentem z numerem indeksu {NumerIndeksu}");
    }
}

// Przykład użycia
class Program
{
    static void Main(string[] args)
    {
        Student student = new Student
        {
            Imie = "Jan",
            Nazwisko = "Kowalski",
            NumerIndeksu = "12345"
        };

        // Wywołanie metody z klasy bazowej
        student.PrzedstawSie();
        // Wywołanie metody z klasy pochodnej
        student.PrzedstawSieJakoStudent();
    }
}

```

Metody wirtualne

Metody wirtualne to metody, które mogą być nadpisywane w klasach pochodnych. Umożliwiają one polimorfizm, czyli zdolność obiektów do przyjmowania różnych form. Metoda wirtualna jest deklarowana w klasie bazowej za pomocą słowa kluczowego `virtual`, a nadpisywana w klasie pochodnej za pomocą słowa kluczowego `override`.

Przykład 2. Metody wirtualne

```

// Klasa bazowa
public class Zwierze
{
    // Metoda wirtualna
    public virtual void WydajDzwiek()
    {
        Console.WriteLine("Zwierzę wydaje dźwięk");
    }
}

// Klasa pochodna
public class Pies : Zwierze
{
    // Nadpisanie metody wirtualnej
    public override void WydajDzwiek()
    {
        Console.WriteLine("Pies szczeka");
    }
}

// Przykład użycia
class Program
{
    static void Main(string[] args)
    {
        Zwierze zwierze = new Zwierze();
        zwierze.WydajDzwiek(); // Wywołanie metody z klasy bazowej

        Zwierze pies = new Pies();
        pies.WydajDzwiek(); // Wywołanie nadpisanej metody z klasy pochodnej
    }
}

```

Klasy abstrakcyjne

Klasa abstrakcyjna to klasa, która nie może być instancjonowana i służy jako klasa bazowa dla innych klas. Może zawierać zarówno metody abstrakcyjne (bez implementacji), jak i metody zdefiniowane. Metody abstrakcyjne muszą być nadpisane w klasach pochodnych.

Przykład 3. Klasy abstrakcyjne

```
// Klasa abstrakcyjna
public abstract class Figura
{
    // Metoda abstrakcyjna
    public abstract double ObliczPole();
}

// Klasa pochodna
public class Kolo : Figura
{
    public double Promien { get; set; }

    // Implementacja metody abstrakcyjnej
    public override double ObliczPole()
    {
        return Math.PI * Promien * Promien;
    }
}

// Przykład użycia
class Program
{
    static void Main(string[] args)
    {
        Figura kolo = new Kolo { Promien = 5 };
        double pole = kolo.ObliczPole();
        Console.WriteLine($"Pole koła: {pole}");
    }
}
```

Zadania do samodzielnego rozwiązania

Zadanie 1. Klasy bazowe i pochodne: Utwórz klasę Pracownik, która dziedziczy po klasie Osoba. Dodaj właściwość Stanowisko i metodę PrzedstawSieJakoPracownik, która wypisuje imię, nazwisko i stanowisko pracownika.

Zadanie 2. Metody wirtualne: Utwórz klasę Kot, która dziedziczy po klasie Zwierze i nadpisuje metodę WydadzDzwiek, aby wypisywała "Kot miauczy". Zmodyfikuj kod w Main, aby utworzyć obiekt klasy Kot i wywołać metodę WydadzDzwiek.

Zadanie 3. Klasy abstrakcyjne: Utwórz klasę Prostokat, która dziedziczy po klasie Figura i implementuje metodę ObliczPole, aby obliczała pole prostokąta. Dodaj odpowiednie właściwości Dlugosc i Szerokosc. Zmodyfikuj kod w Main, aby utworzyć obiekt klasy Prostokat i obliczyć jego pole.